

BAB IV

HASIL DAN PEMBAHASAN

4.1. Perencanaan

Tahap perencanaan menghasilkan rancangan arsitektur sistem IoT yang akan diimplementasikan. Tahap ini ditetapkan tiga skenario untuk mengevaluasi pengaruh perbedaan arsitektur dan protokol terhadap kinerja sistem. Skenario pertama mengombinasikan arsitektur fog computing dengan protokol CoAP untuk komunikasi antara mikrokontroler dan server, serta protokol MQTT untuk komunikasi antara website dengan server. Skenario kedua mengombinasikan arsitektur fog computing dengan protokol MQTT untuk seluruh komunikasi pada sistem IoT. Skenario ketiga mengombinasikan arsitektur *fog computing* dengan protokol CoAP untuk komunikasi antara mikrokontroler dan server, serta protokol HTTP untuk komunikasi antara *website* dengan server. Ketiga skenario ini menerapkan *fuzzy logic* untuk melakukan pengolahan data sensor untuk menentukan aksi aktuator. Selain itu untuk himpunan *fuzzy logic* berdasarkan referensi penelitian Irwanto et al (2024) yang disesuaikan dengan karakteristik nilai ADC dari sensor yang digunakan pada penelitian ini.

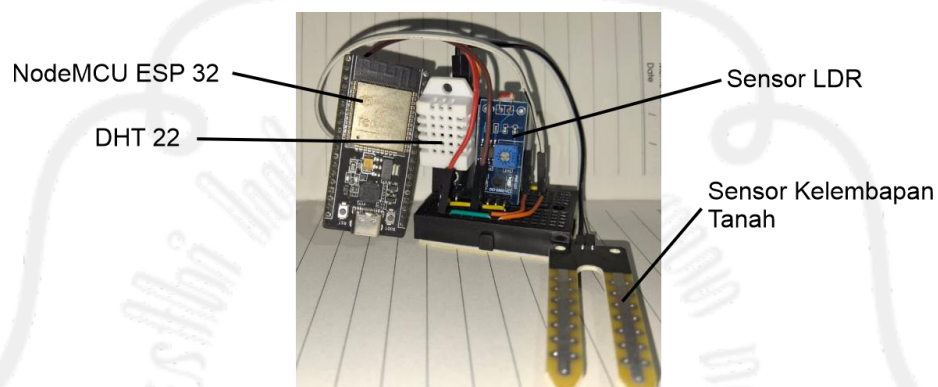
4.2. Perancangan Prototipe

Implementasi sistem dibangun berdasarkan tiga skenario yang telah dirancang pada bab metodologi, yaitu skenario pertama mikrokontroler ESP32 berkomunikasi dengan *fog node* menggunakan protokol CoAP, sedangkan komunikasi antara *fog node* dengan pengguna menggunakan protokol MQTT melalui *cloud broker*. Skenario kedua mikrokontroler mengirim data langsung ke *cloud broker* MQTT tanpa melalui *fog node*, sehingga pemrosesan dilakukan pada cloud. Skenario ketiga mikrokontroler berkomunikasi dengan *fog node* menggunakan CoAP, sedangkan *fog node* terhubung ke aplikasi web melalui HTTP REST API.

Backend menyediakan beberapa endpoint berbasis REST API dengan metode GET, POST, dan PUT untuk menangani pengambilan data sensor dan penyimpanan riwayat. Metode GET digunakan untuk mengambil riwayat data sensor dan aktuator yang tersimpan dalam basis data, seperti endpoint `/api/sensor/latest` dan `/api/kontrol/latest`. Metode POST digunakan untuk mengirimkan data dari sistem backend, seperti data kontrol pengguna pada endpoint `/api/user/add` yang selanjutnya

akan disimpan ke dalam basis data. Metode PUT digunakan untuk memperbarui status pengendalian aktuator, baik pada mode otomatis maupun manual, melalui endpoint `/api/kontrol/otomatis/:id` dan `/api/kontrol/update/:id`, sehingga sistem dapat menyesuaikan kondisi aktuator sesuai dengan hasil pemrosesan data.

Implementasi perangkat keras meliputi sensor DHT22 untuk mengukur suhu dan kelembapan udara, sensor LDR untuk mengukur intensitas cahaya, dan *soil moisture sensor* untuk mengukur kelembapan tanah. Aktuator terdiri dari pompa air, kipas angin, dan lampu LED yang diatur secara otomatis berdasarkan hasil inferensi *fuzzy logic*. Prototipe perangkat keras dapat dilihat pada Gambar 4.1 dan Gambar 4.2.



Gambar 4.1. Prototipe ESP 32 dengan Sensor



Gambar 4.2. Prototipe ESP 32 dengan Aktuator

4.3. Hasil Pengujian Sensor dan Aktuator

Pengujian sensor dan aktuator dilakukan untuk memastikan integrasi antara komponen fisik pada mikrokontroler dan *fuzzy logic* bekerja dengan baik. Data dari

simulasi data sensor dan kontrol aktuator dari *fuzzy logic* dapat dilihat pada Tabel 4.2. Selain itu, contoh perhitungan *fuzzy logic* yang digunakan pada sistem ini adalah sebagai berikut untuk menggambarkan proses fuzzifikasi, inferensi dan penentuan output kontrol dengan parameter suhu udara 31,3 °C, kelembapan udara 78,4%, intensitas cahaya 434 dan kelembapan tanah 1130.

Proses pertama yaitu fuzzifikasi, yaitu untuk mengubah nilai input sensor menjadi derajat keanggotaan pada setiap himpunan *fuzzy*. Setiap parameter atau variabel memiliki tiga kategori dengan fungsi derajat keanggotaan segitiga (a, b, c) seperti ditunjukkan pada Tabel 4.1. Penggunaan keanggotaan segitiga karena bentuknya sederhana dan efisien secara komputasi, sehingga sesuai untuk sistem IoT yang dijalankan pada fog node. Fungsi segitiga juga mampu merepresentasikan perubahan nilai sensor secara bertahap dan lebih responsif terhadap perubahan input.

Tabel 4.1. Keanggotaan Fuzzy

Variabel	Kategori Fuzzy	Parameter (a, b, c)
Suhu (°C)	Rendah	(0, 0, 20)
	Sedang	(15, 22.5, 30)
	Tinggi	(25, 35, 45)
Kelembapan Udara (%)	Rendah	(0, 30, 50)
	Sedang	(40, 60, 80)
	Tinggi	(70, 100, 100)
Cahaya (ADC)	Rendah	(0, 0, 1365)
	Sedang	(1200, 2000, 2800)
	Tinggi	(2600, 4095, 4095)
Kelembapan Tanah (ADC)	Rendah	(0, 0, 1365)
	Sedang	(1200, 2000, 2800)
	Tinggi	(2600, 4095, 4095)

a. Suhu Udara

- Suhu rendah $\rightarrow 31,3 > 20 = 0$
- Suhu sedang $\rightarrow 31,3 > 30 = 0$

- Suhu tinggi

$$\mu = \frac{31,3 - 25}{35 - 25} = 0,63$$

b. Kelembapan udara

- Kelembapan udara rendah $\rightarrow 78,4 > 50 = 0$
- Kelembapan udara sedang

$$\mu = \frac{80 - 78,4}{80 - 60} = 0,08$$

- Kelembapan udara tinggi

$$\mu = \frac{78,4 - 70}{100 - 70} = 0,28$$

c. Intensitas Cahaya

- Intensitas cahaya rendah

$$\mu = \frac{1365 - 434}{1365 - 0} = 0,682$$

- Intensitas cahaya sedang $\rightarrow 434 > 1200 = 0$
- Intensitas cahaya tinggi $\rightarrow 434 > 2600 = 0$

d. Kelembapan Tanah

- Kelembapan tanah rendah

$$\mu = \frac{1365 - 1130}{1365 - 0} = 0,172$$

- Kelembapan tanah sedang $\rightarrow 1130 > 1200 = 0$
- Kelembapan tanah tinggi $\rightarrow 1130 > 2600 = 0$

Tahap inferensi sistem melakukan proses penalaran berdasarkan derajat keanggotaan hasil fuzzifikasi untuk menentukan output aktuator yang aktif. Setiap aktuator memiliki aturan sesuai pada Lampiran 1. Setiap aktuator dapat diaktifkan oleh lebih dari satu aturan, maka hasil akhirnya diambil dari nilai maksimum keanggotaan yang diperoleh dari semua aturan yang relevan. Berdasarkan hasil fuzzifikasi dari kondisi input yang diberikan, diperoleh nilai keanggotaan untuk himpunan *fuzzy*. Kelembapan tanah memiliki nilai ADC 1130 didapatkan bahwa tingkat keanggotaannya dalam kategori sedang adalah 0 dan kategori tinggi adalah nol. Sementara itu, untuk Suhu udara senilai 31.3 tingkat keanggotaannya kategori tinggi

adalah 0,63. Intensitas cahaya memiliki nilai ADC 434 tingkat keanggotaannya dalam kategori tinggi bernilai nol. Kelembapan udara sebesar 78,4% memiliki derajat keanggotaan sebesar 0 pada kategori rendah, 0,08 pada kategori sedang dan 0,28 pada kategori tinggi.. Maka nilai aktivasi tiap aktuator adalah:

Penyiram

$$\alpha_{\text{penyiram}} = 0$$

Kipas Angin

$$\alpha_{\text{kipas}} = 0,63$$

Lampu LED

$$\alpha_{\text{lampu}} = \max(\mu_{\text{cahaya tinggi}}, \mu_{\text{kelembaban udara rendah}}) = 0$$

Penentuan keputusan akhir menggunakan nilai ambang batas 0,5. Nilai ambang 0,5 dipilih karena merepresentasikan titik tengah derajat keanggotaan yang menunjukkan kondisi transisi antara *OFF* dan *ON*, sehingga keputusan aktuator tidak terlalu sensitif terhadap fluktuasi kecil nilai inferensi. Jika hasil inferensi melebihi ambang batas maka aktuator akan bernilai *ON*, sebaliknya jika hasil inferensi kurang atau sama dengan ambang batas maka aktuator akan bernilai *OFF*. Berdasarkan hasil inferensi penyiram yaitu 0 maka aktuator penyiram bernilai *OFF*. Hasil inferensi kipas yaitu 0,63 maka aktuator kipas akan bernilai *ON*. Hasil inferensi lampu yaitu 0 maka aktuator lampu akan bernilai *OFF*.

Tabel 4.2. Contoh Hasil Pengujian *Fuzzy Logic*

Parameter				Nilai Fuzzy			Aktuator
Kelembapan Tanah	Suhu Udara	Kelembapan Udara	Intensitas Cahaya	Penyiram	Kipas	Lampu	
1119	18	33	912	0	0	0.85	Penyiram <i>OFF</i> , Kipas <i>OFF</i> , Lampu <i>ON</i>
1119	28,2	33	912	0	0,32	0.85	Penyiram <i>OFF</i> , Kipas <i>OFF</i> , Lampu <i>ON</i>
1119	31.3	33	912	0	0,63	0.85	Penyiram <i>OFF</i> , Kipas <i>ON</i> , Lampu <i>ON</i>
2101	15	55,4	434	0,87	0	0	Penyiram <i>ON</i> , Kipas <i>OFF</i> , Lampu <i>OFF</i>
2101	15	81,1	2736	0,87	0	0,09	Penyiram <i>ON</i> , Kipas <i>OFF</i> , Lampu <i>OFF</i>
3469	18	35	1651	0,58	0	0.75	Penyiram <i>ON</i> , Kipas <i>OFF</i> , Lampu <i>ON</i>
3549	19	31	3874	0,63	0	0,95	Penyiram <i>ON</i> , Kipas <i>OFF</i> , Lampu <i>ON</i>

Hasil pada Tabel 4.2 menunjukkan integrasi antara mikrokontroler dan *fuzzy logic* dapat berjalan dengan baik sesuai dengan aturan yang telah dibuat.

4.4. Hasil Pengujian Kinerja Sistem

Pengujian kinerja sistem untuk mengevaluasi parameter QoS meliputi latensi, *jitter* dan *packet loss* pada ketiga skenario arsitektur. Pengujian dilakukan selama 15 menit dengan interval pengiriman data oleh sensor setiap 15 detik. Hasil pengujian pengiriman dari mikrokontroler ke server dapat dilihat pada Lampiran 3, Lampiran 5, dan Lampiran 7 dan hasil pengujian mikrokontroler ke pengguna atau website dapat dilihat pada Lampiran 4, Lampiran 6, dan Lampiran 8. Selain itu, proses perhitungan latensi, *jitter* dan *packet loss* pada Lampiran 3 dijelaskan dalam proses berikut ini.

Latensi

$$\begin{aligned} \text{rata - rata latensi} &= \frac{102 + 102 + 101 + \dots + 113 + 111 + 112}{60} \\ \text{rata - rata latensi} &= 107,35 \end{aligned}$$

Jitter

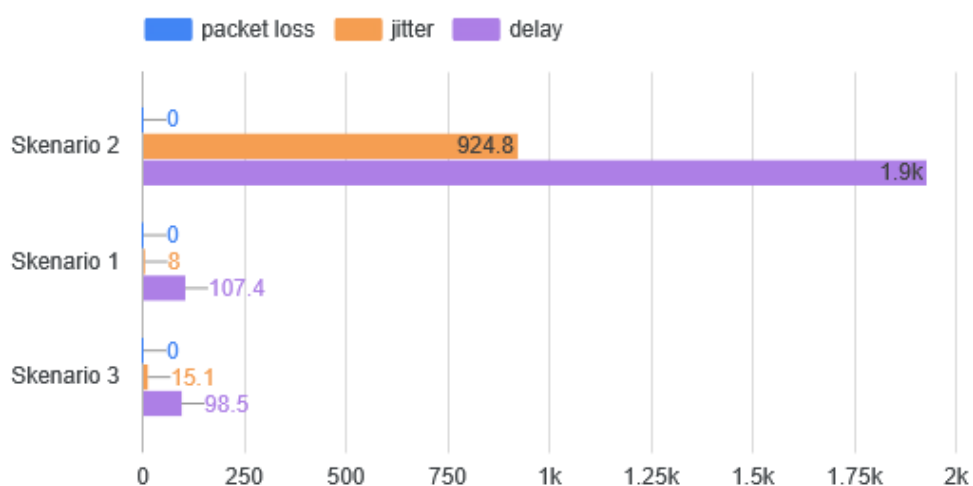
$$\begin{aligned} \sigma &= \sqrt{\frac{(102 - 98,52)^2 + (101 - 98,52)^2 + \dots + (112 - 98,52)^2}{60}} \\ \sigma &= 8,02 \end{aligned}$$

Packet loss

$$\begin{aligned} \text{Packet loss} &= \frac{\text{Jumlah paket yang hilang}}{\text{Total paket yang dikirim}} \times 100\% \\ \text{Packet loss} &= \frac{0}{60} \times 100\% \\ \text{Packet loss} &= 0 \end{aligned}$$

Tabel 4.3. Hasil Pengujian Kinerja Sistem Mikrokontroler ke Server

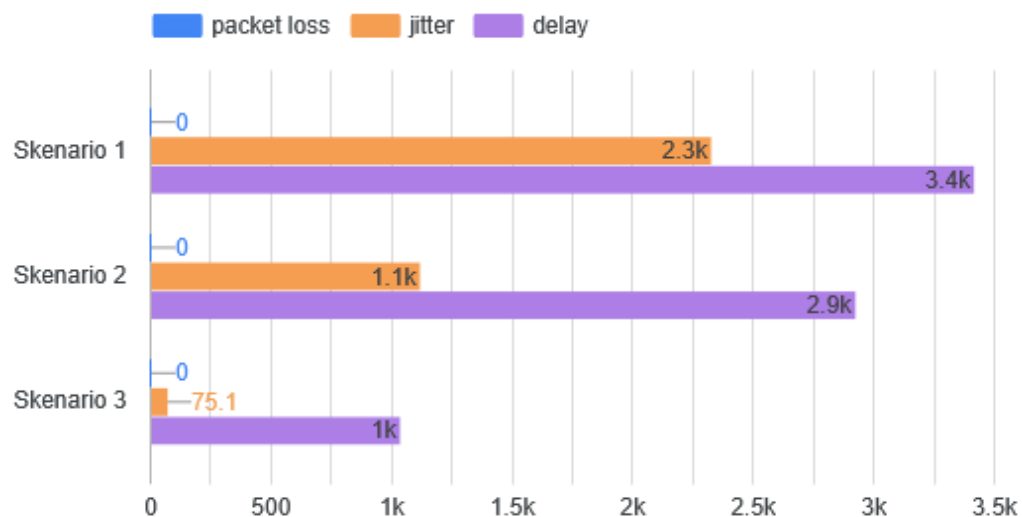
Skenario	Arsitektur	Protokol	Latensi (ms)	Jitter (ms)	Packet loss (%)
1	Fog Computing	CoAP + MQTT	107,35	8,02	0
2	Cloud Computing	MQTT	1929,77	924,8	0
3	Fog Computing	CoAP + HTTP	98,52	15,13	0



Gambar 4.3. Grafik Pengujian Kinerja Sistem Mikrokontroler ke Server

Tabel 4.4. Hasil Pengujian Kinerja Sistem Mikrokontroler ke Website

Skenario	Arsitektur	Protokol	Latensi (ms)	Jitter (ms)	Packet loss (%)
1	Fog Computing	CoAP + MQTT	3419,42	2329,26	0
2	Cloud Computing	MQTT	2927,57	1121,4	0
3	Fog Computing	CoAP + HTTP	1038,88	75,09	0



Gambar 4.4. Grafik Pengujian Kinerja Sistem Mikrokontroler ke Website

Berdasarkan hasil pengujian kinerja sistem dari mikrokontroler ke server yang diperoleh pada Tabel 4.3 dan Gambar 4.3 menunjukkan skenario pertama memiliki latensi rata-rata 107,52 ms dan jitter 8,02 ms. Skenario kedua memiliki latensi rata-rata 1929,767 ms dan jitter 924,7994 ms, sedangkan skenario ketiga memiliki latensi rata-rata 98,52 ms dan jitter 15,13 ms. Ketiga skenario menunjukkan nilai *packet loss* sebesar 0% yang artinya tidak ada kehilangan paket selama proses transmisi data dari mikrokontroler ke server. Nilai yang dicetak tebal pada Tabel 4.3 merepresentasikan hasil yang terbaik.

Hasil pengujian kinerja sistem secara keseluruhan dari mikrokontroler sampai pengguna yang diperoleh pada Tabel 4.4 dan Gambar 4.4 menunjukkan skenario pertama memiliki latensi rata-rata sebesar 3419,417 ms dan *jitter* 2329,263. Skenario kedua memiliki latensi rata-rata 2927,567 ms dan *jitter* 1121,402 ms, sedangkan skenario ketiga memiliki latensi rata-rata sebesar 1038,883 ms dan *jitter* sebesar 75,08642 ms. Ketiga skenario memiliki *packet loss* sebesar 0% yang artinya tidak ada paket yang hilang selama transmisi data dari mikrokontroler sampai ke pengguna. Nilai yang dicetak tebal pada Tabel 4.4 merepresentasikan hasil yang terbaik.

4.5. Analisa Perbandingan

Hasil pada Tabel 4.3 dan Tabel 4.4 menunjukkan arsitektur *fog computing* dan protokol CoAP memberikan latensi yang lebih rendah dan jitter yang lebih stabil dibandingkan dengan sistem berbasis cloud dan protokol MQTT. Pada dua skenario

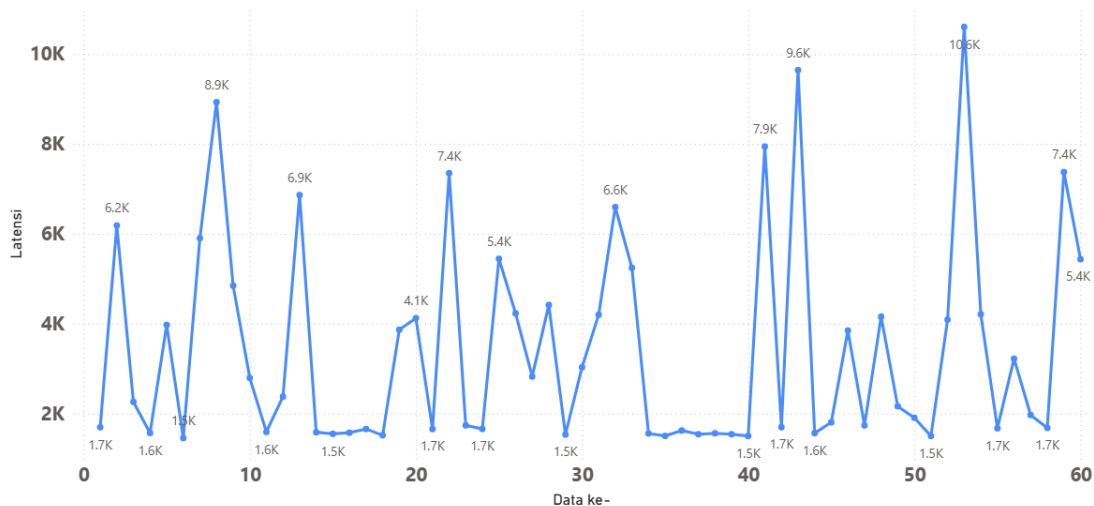
yaitu skenario pertama dan skenario ketiga pemrosesan data dilakukan di *fog node* yang berada di jaringan lokal untuk melakukan proses pengolahan data. Kondisi ini menyebabkan latensi transmisi data antara mikrokontroler dan pengguna dapat berkurang dibandingkan dengan sistem berbasis cloud, karena jarak komunikasi dan jumlah hop jaringan yang harus dilalui paket data menjadi lebih sedikit. Selain itu, jitter pada arsitektur *fog computing* lebih rendah dibandingkan arsitektur *cloud*, hal ini menunjukkan bahwa waktu pengiriman data menjadi lebih stabil dan sistem mampu memberikan respons yang cepat serta konsisten.

Skenario pertama dan ketiga, keduanya sama-sama menggunakan CoAP untuk komunikasi antara mikrokontroler dan *fog node*, namun pada komunikasi antara mikrokontroler dan pengguna memiliki perbedaan. Pada skenario pertama menggunakan MQTT dan skenario ketiga menggunakan HTTP. Skenario ketiga yang menggunakan HTTP menunjukkan latensi lebih rendah dan jitter lebih stabil dibandingkan skenario pertama yang menggunakan MQTT. Skenario pertama *fog node* mengirimkan data ke *cloud broker* MQTT terlebih dahulu kemudian *website* mengambil data dari *cloud broker* MQTT. Penambahan cloud broker ini menambah overhead komunikasi dan meningkatkan kemungkinan terjadinya keterlambatan akibat fluktuasi jaringan internet. Sebaliknya skenario ketiga, *website* mengambil data secara langsung dari *fog node* menggunakan protokol HTTP, sehingga jalur komunikasi menjadi lebih pendek dan waktu respons sistem menjadi lebih cepat dan konsisten.

Skenario kedua menggunakan *cloud computing* dengan protokol MQTT menghasilkan latensi yang tinggi dibandingkan skenario lainnya. Hal ini karena seluruh proses pengiriman data dilakukan melalui jaringan internet menuju ke broker MQTT. Setiap paket data harus melewati beberapa lapisan komunikasi dan proses *routing* yang lebih panjang, serta dipengaruhi juga oleh kestabilan jaringan internet pada sistem. Hal ini menyebabkan waktu pengiriman lebih lama dan stabilitas waktu antar paket rendah.

Berdasarkan Gambar 4.5 yang merujuk pada Lampiran 4, terlihat bahwa grafik latensi pada skenario pertama menunjukkan pola yang tidak stabil dengan fluktuasi yang cukup tajam. Terdapat beberapa lonjakan latensi yang tinggi yang mengindikasikan adanya gangguan atau ketergantungan terhadap kualitas koneksi

internet. Ketidakstabilan ini terjadi pada skenario pertama karena proses komunikasi masih bergantung pada jaringan internet pada tahap pengiriman data dari *server* ke *website* yang menggunakan protokol MQTT, sehingga terjadi peningkatan waktu pengiriman paket secara signifikan. Kondisi tersebut menyebabkan nilai rata-rata latensi pada skenario pertama menjadi lebih tinggi dibandingkan skenario kedua.



Gambar 4.5. Grafik Latensi Skenario 1 ESP32 ke Website

Perbandingan ketiga skenario membuktikan kombinasi *fog computing* dan protokol CoAP memberikan optimalisasi dalam komunikasi data IoT. Keunggulan utamanya terletak pada pemrosesan lokal yang mengurangi ketergantungan pada jaringan internet, serta protokol CoAP yang cepat. Sehingga sistem memiliki latensi yang rendah dan stabilitas waktu yang tinggi.

Sistem juga berhasil menerapkan *fuzzy logic* untuk mengendalikan aktuator secara otomatis. Berdasarkan hasil pada Tabel 4.2 sistem mampu merespons kondisi lingkungan sesuai dengan aturan *fuzzy* yang telah dibuat. Pompa air akan aktif saat kelembapan tanah rendah, kipas akan menyala saat suhu udara tinggi, dan lampu LED akan hidup saat intensitas cahaya rendah. Hal ini membuktikan bahwa algoritma *fuzzy logic* dapat berjalan dengan baik dalam sistem dan mampu menyesuaikan keputusan aktuator berdasarkan parameter kondisi lingkungan yang terjadi secara *real time*.